LABORATORY OBSERVATION

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 2
Student Name: G.Kavya
Roll No.: A24126552082
Date: 30-08-2026

TITLE

Student Profile Manager Using JavaScrip

AIM

To create a Student Profile Manager using JavaScript that accepts user-provided student details and dynamically creates and displays the student profile on the webpage using DOM manipulation.

OBJECTIVES

- To create a JavaScript Student class with properties such as Name, Roll Number, Department, and CGPA.
- To create an object of the Student class using user-provided values.
- To use DOM selection to access webpage elements.
- To dynamically create and display the student profile using DOM manipulation.
- To provide a button for generating the student profile.
- To apply basic CSS styling to the dynamically generated profile.

TASK DESCRIPTION

Create a Student Profile Manager using JavaScript.

The application should:

- Create a JavaScript Student class with properties such as:
  - Name
  - Roll Number
  - Department
  - CGPA
- Create an object of the Student class using user-provided values.
- Use DOM manipulation to display the student details dynamically on the webpage.

- Provide a button to generate the profile.
- When the button is clicked, dynamically create and display the student profile using DOM manipulation.
- Apply basic CSS styling to the dynamically generated profile.

ALGORITHM / PROCEDURE

1. Create an HTML document with input fields for Name, Roll Number, Department, and CGPA.
2. Add a button to generate the student profile.
3. Create a JavaScript Student class with the required properties.
4. Define a constructor to initialize the student details.
5. Select the student form and profile output elements using getElementById().
6. Handle the button/form submission using an event listener.
7. Read the user-provided values using FormData.
8. Remove unnecessary spaces using trim().
9. Format the CGPA to two decimal places using toFixed(2).
10. Create a Student object using the entered values.
11. Clear any previously generated profile.
12. Dynamically create the profile heading and detail elements.
13. Modify the content of the created elements using textContent.
14. Add the generated elements to the webpage.
15. Apply CSS styling to the form and generated profile.

PROGRAM / SOURCE CODE — Index.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Student Profile Manager</title>
    <link rel="stylesheet" href="styles.css" />
  </head>
  <body>
    <main class="profile-manager">
      <h1>Student Profile Manager</h1>
      <form id="studentForm" class="student-form">
        <label for="name">Name</label>
        <input id="name" name="name" type="text" value="Ravi Kumar" required />

        <label for="rollNumber">Roll Number</label>
        <input id="rollNumber" name="rollNumber" type="text" value="A23126552001" required />

        <label for="department">Department</label>
        <input id="department" name="department" type="text" value="CSE (AI & ML)" required />

        <label for="cgpa">CGPA</label>
        <input id="cgpa" name="cgpa" type="number" min="0" max="10" step="0.01" value="8.42" required />

        <button type="submit">Generate Profile</button>
      </form>

      <section id="profileOutput" class="profile-output" aria-live="polite"></section>
    </main>

    <script src="script.js"></script>
  </body>
</html>
```

Styles.css

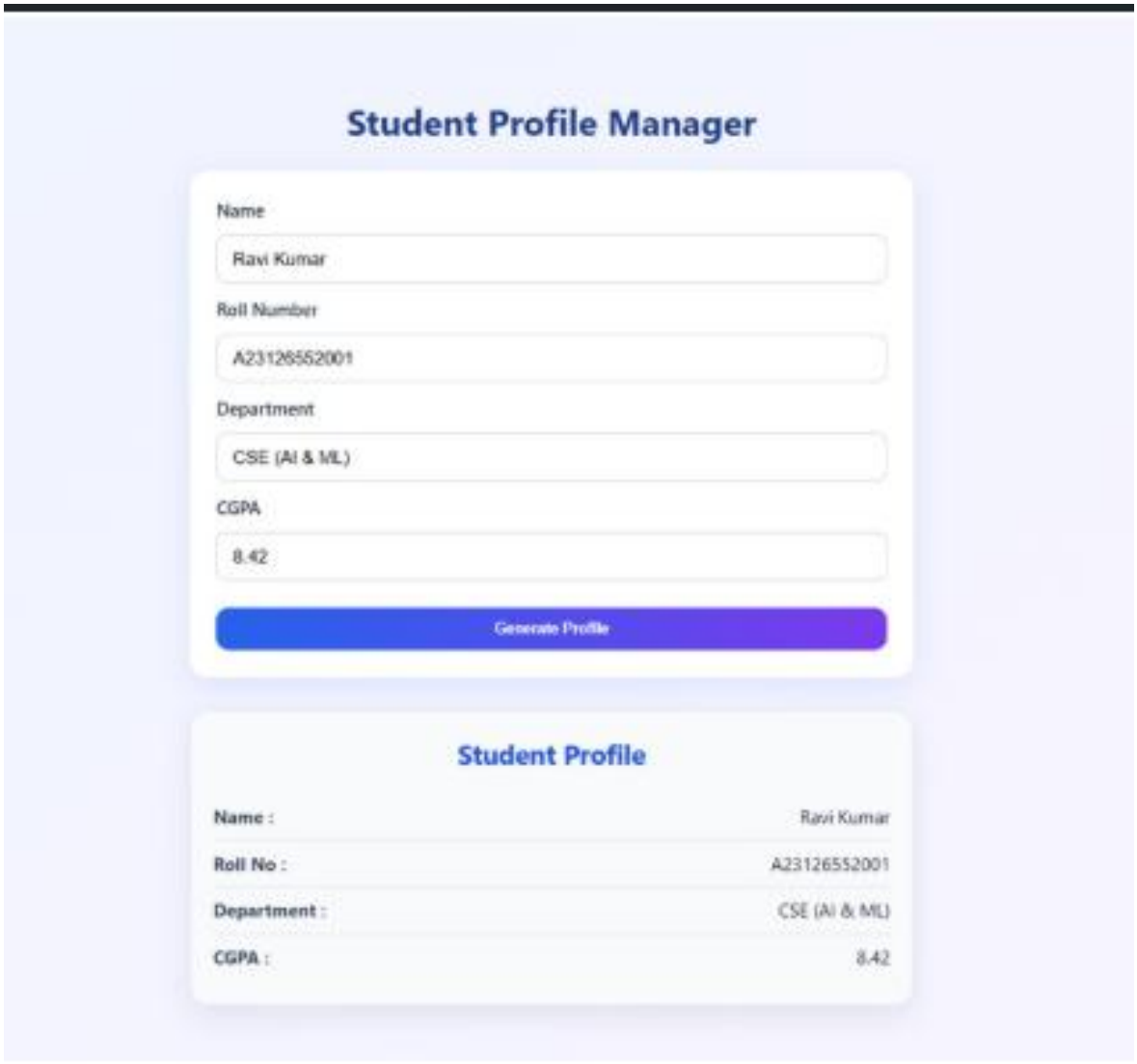
```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  min-height: 100vh;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 32px 16px 40px;
  font-family: Georgia, 'Times New Roman', serif;
  background: #f5f5ff;
  color: #1f2937;
}

.profile-manager {
  width: min(488px, 100%);
}

h1 {
  margin: 0 0 26px;
  color: #214787;
  text-align: center;
  font-family: Arial, sans-serif;
  font-size: 1.7rem;
  line-height: 1.1;
}

.student-form {
  display: flex;
  flex-direction: column;
  gap: 7px;
  padding: 20px 18px 18px;
  background: #ffffff;
  border-radius: 12px;
  box-shadow: 0 8px 20px rgba(70, 86, 140, 0.12);
}

.student-form label {
  align-self: flex-start;
  font-family: Arial, sans-serif;
  font-size: 0.8rem;
  font-weight: bold;
}

.student-form input {
  width: 100%;
  padding: 8px 10px;
  border: 1px solid #d9dde5;
  border-radius: 7px;
  color: #26313f;
  font-family: Arial, sans-serif;
  font-size: 0.8rem;
}

.student-form input:focus {
  outline: 2px solid #b6c8ff;
  border-color: #5a78e8;
}

.student-form button {
  width: 100%;
  margin-top: 11px;
  padding: 8px 18px;
  border: 0;
  border-radius: 7px;
  background: linear-gradient(90deg, #2563eb, #7c2fe5);
  color: #ffffff;
  cursor: pointer;
  font-family: Arial, sans-serif;
  font-size: 0.75rem;
  font-weight: bold;
}

.student-form button:hover {
  opacity: 0.9;
}

.profile-output {
  margin-top: 22px;
```

```
  padding: 20px 16px 18px;
  background: #ffffff;
  border-radius: 12px;
  box-shadow: 0 8px 20px rgba(70, 86, 140, 0.12);
}

.profile-output:empty {
  display: none;
}

.profile-output h2 {
  margin: 0 0 14px;
  color: #214787;
  text-align: center;
  font-family: Arial, sans-serif;
  font-size: 1.2rem;
}

.profile-output dl {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 0;
  margin: 0;
  font-family: Arial, sans-serif;
  font-size: 0.78rem;
}

.profile-output dt {
  padding: 8px 0;
  border-bottom: 1px solid #e6e9ef;
  font-weight: bold;
}

.profile-output dd {
  margin: 0;
  padding: 8px 0;
  border-bottom: 1px solid #e6e9ef;
  color: #315d58;
  text-align: right;
}

@media (max-width: 560px) {
  .profile-manager {
    width: min(488px, 100%);
  }

  .student-form {
    padding-inline: 18px;
  }
}
```

Script.js

```
class Student {
  constructor(name, rollNumber, department, cgpa) {
    this.name = name;
    this.rollNumber = rollNumber;
    this.department = department;
    this.cgpa = cgpa;
  }
}

const studentForm = document.getElementById('studentForm');
const profileOutput = document.getElementById('profileOutput');

studentForm.addEventListener('submit', (event) => {
  event.preventDefault();

  const formData = new FormData(studentForm);

  const student = new Student(
    formData.get('name').trim(),
    formData.get('rollNumber').trim(),
    formData.get('department').trim(),
    Number(formData.get('cgpa')).toFixed(2)
  );

  profileOutput.replaceChildren();

  const heading = document.createElement('h2');
  heading.textContent = 'Student Profile';

  const details = document.createElement('dl');

  const profileDetails = [
    ['Name', student.name],
    ['Roll No', student.rollNumber],
    ['Department', student.department],
    ['CGPA', student.cgpa]
  ];

  profileDetails.forEach(([label, value]) => {
    const term = document.createElement('dt');
    term.textContent = label;

    const description = document.createElement('dd');
    description.textContent = value;

    details.append(term, description);
  });

  profileOutput.append(heading, details);
});
```

OUTPUT:


Student Profile Manager
Name
Ravi Kumar
Roll Number
A23126552001
Department
CSE (AI & ML)
CGPA
8.42
Generate Profile
Student Profile
Name : Ravi Kumar
Roll No : A23126552001
Department : CSE (AI & ML)
CGPA : 8.42

CODE EXPLANATION

Component	Purpose
class Student	Creates a Student class.
constructor()	Initializes student details.
this.name	Stores the name.
this.rollNumber	Stores the roll number.
this.department	Stores the department.
this.cgpa	Stores the CGPA.
getElementById()	Selects HTML elements from the webpage.
addEventListener()	Handles the form submission event.
FormData	Reads values entered by the user.
.trim()	Removes unnecessary spaces.
toFixed(2)	Formats the CGPA to two decimal places.
replaceChildren()	Clears the previously generated profile.
createElement()	Creates HTML elements dynamically.
textContent	Modifies the content of elements.
append()	Adds created elements to the webpage.
:hover	Adds an effect when the mouse is over the button.
@media	Provides responsive styling for smaller screens.

TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Observed Result / Status
TC-01	Open the webpage	Student Profile Manager should be displayed	Page displayed correctly — Pass
TC-02	Enter valid student details	Student profile should be generated	Profile generated correctly — Pass
TC-03	Enter CGPA as 8.42	CGPA should be displayed with two decimal places	8.42 displayed — Pass
TC-04	Click Generate Profile again	Previous profile should be replaced	Profile replaced correctly — Pass
TC-05	Enter different student details	New student profile should be generated	Profile updated correctly — Pass
TC-06	Open webpage on smaller screen	Layout should adjust to screen size	Responsive layout displayed — Pass

RESULT

Thus, the Student Profile Manager was successfully developed using JavaScript. A Student class and object were used to store the student details, and DOM selection, dynamic element creation, content

modification, element insertion, and event handling were successfully demonstrated. Basic CSS styling was applied to the generated profile.